ARRAYS/STRINGS

TWO POINTERS
SLIDING WINDOW
PREFIX SUM

HASHING (SETS & DICTIONARIES)

ANAGRAM GROUPING
FREQUENT MAPS
COUNTING

STACKS/QUEUES

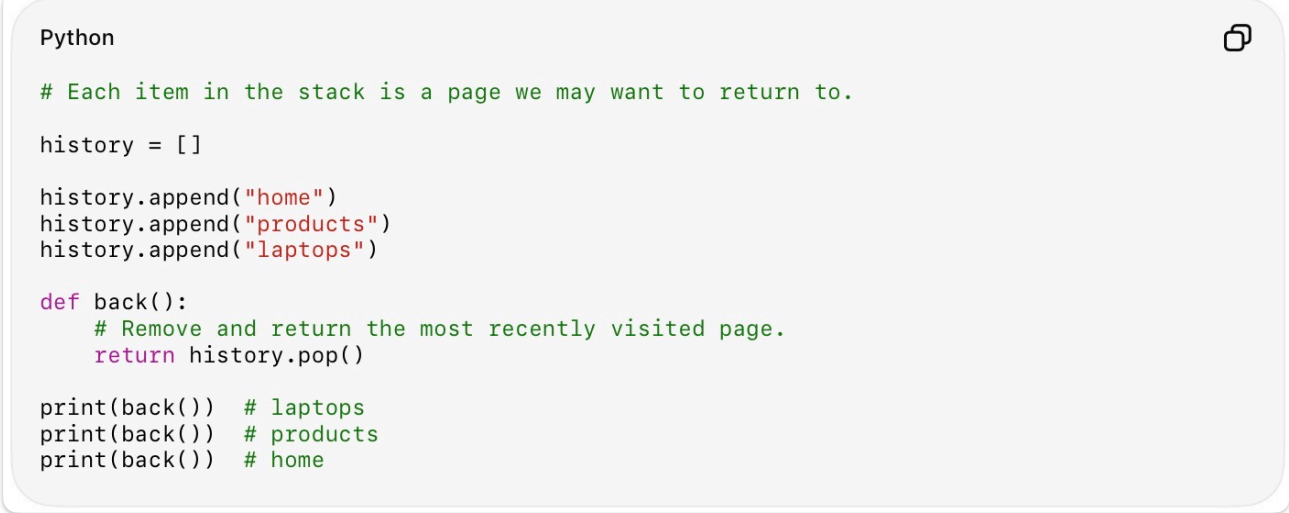
STACK

LIFO

Python
list []

O(n)
BOTTOM ITEM
remove
insert

O(1)
TOP ITEM
change
append(x) push onto
pop() remove
stack[-1] read
BOTTOM ITEM
read stack[0]
LEN()



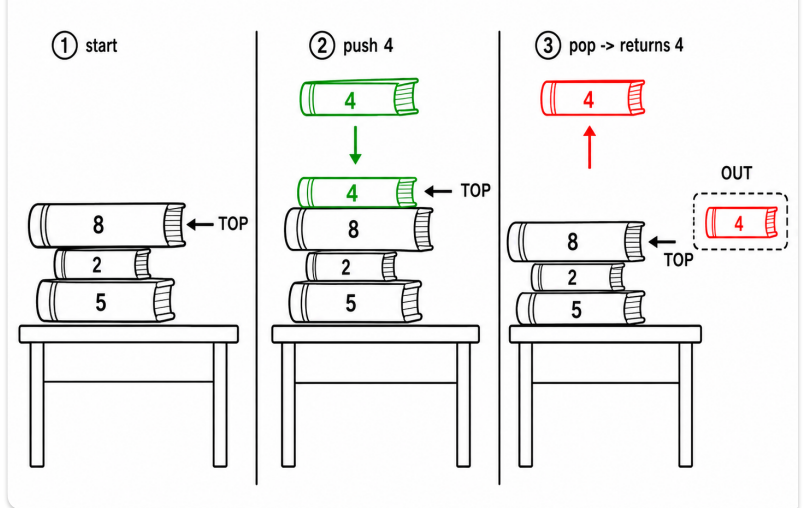
Python

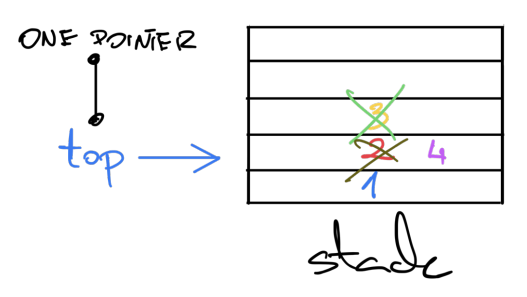
```python
# Each item in the stack is a page we may want to return to.

history = []

history.append("home")
history.append("products")
history.append("laptops")

def back():
    # Remove and return the most recently visited page.
    return history.pop()

print(back())  # laptops
print(back())  # products
print(back())  # home
```

When to think of it

- I need to restore the most recently saved state first.
- I need undo / backtracking behavior.
- I need to return to the latest unfinished or previous context.
- The natural processing order is last-in, first-out.

Only the top item can be pushed or popped.


① start
TOP
8
2
5
② push 4
4
TOP
③ pop -> returns 4
OUT


ONE POINTER
top
4
1
stack

push(1)
push(2)
push(3)
pop()
pop()
push(4)

USE CASES
backtracking
find correct path through maze
depth-first search
undo (pop) / redo (push)

newest element processed first

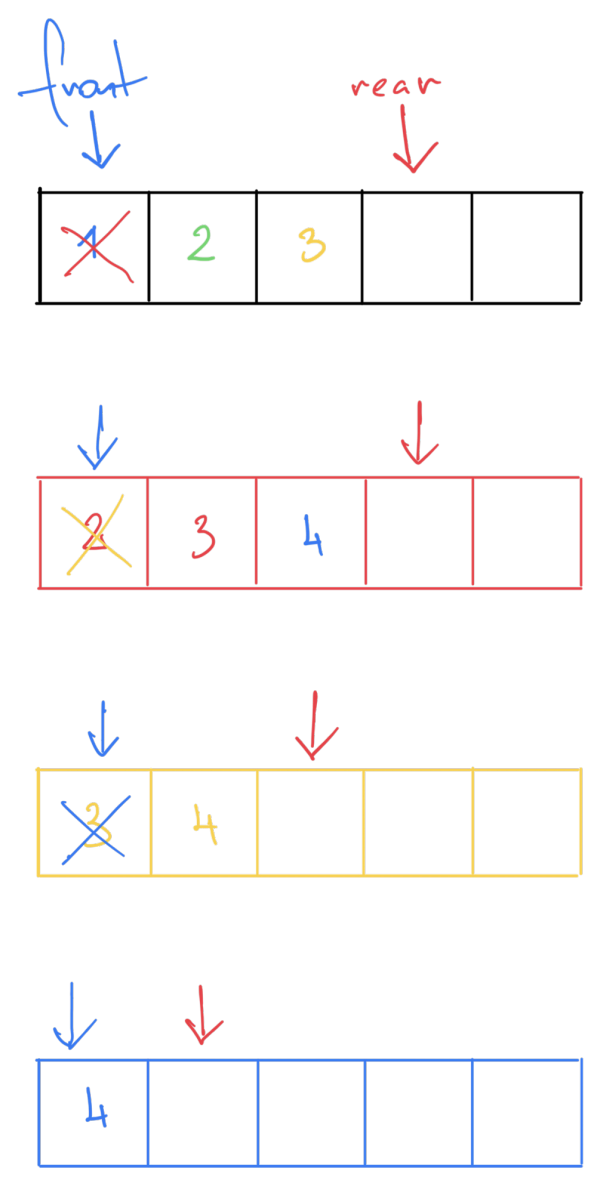
2.9K likes, 90 comments. "Monotonic Stack Data Structure Explained"
AlgoMonster
youtu.be

QUEUE

FIFO

Python
from collections import deque

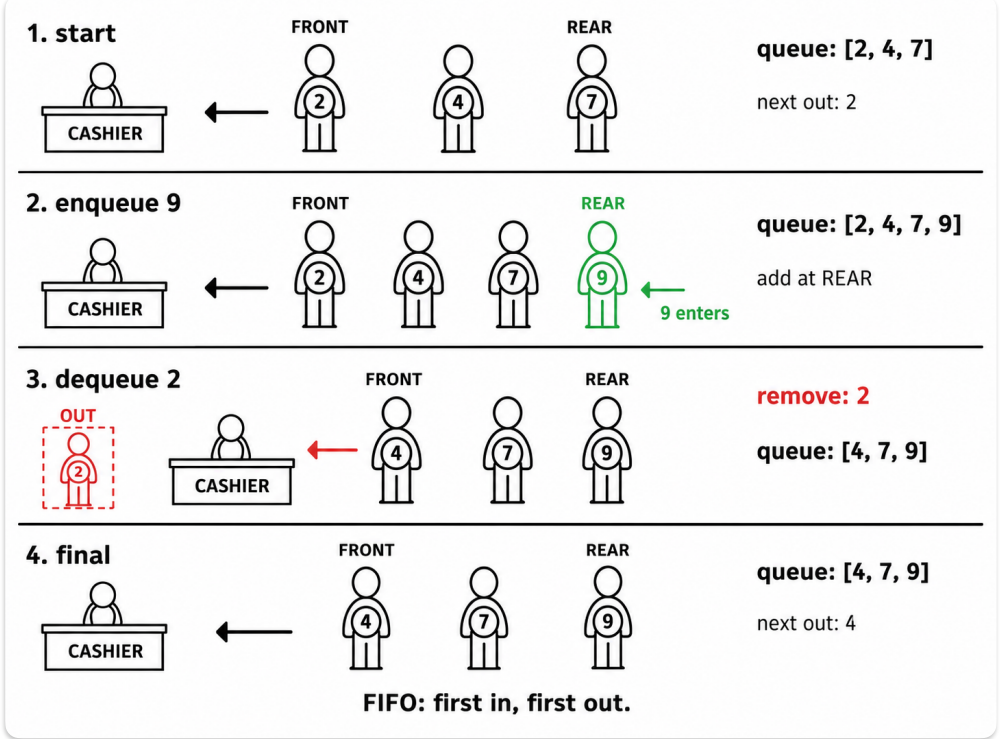
O(1)
FRONT
remove queue.popleft()
read
BACK
add queue.append(x)
read
LEN()


front
rear
1
2
3
2
3
4
2
4
4

enqueue(1)
enqueue(2)
enqueue(3)
dequeue()

enqueue(4)
dequeue()

dequeue()


1. start
FRONT
REAR
CASHIER
queue: [2, 4, 7]
next out: 2
2. enqueue 9
FRONT
REAR
CASHIER
9 enters
queue: [2, 4, 7, 9]
add at REAR
3. dequeue 2
OUT
CASHIER
FRONT
REAR
remove: 2
queue: [4, 7, 9]
4. final
CASHIER
FRONT
REAR
queue: [4, 7, 9]
next out: 4

FIFO: first in, first out.

USE CASES
operating systems
- CPU and disk scheduling
Spotify
- playlist
breadth-first search

oldest element processed first

MONOTHONIC STACK

START
heights = [6, 2, 8, 5, 7]
stack = []
next_higher = [-1, -1, -1, -1, -1]

i = 0, height = 6

stack is empty

push index 0

stack = [0]
stack heights = [6]
next_higher = [-1, -1, -1, -1, -1]

----
i = 1, height = 2

top height = 6

6 < 2 ? no

push index 1

stack = [0, 1]
stack heights = [6, 2]
next_higher = [-1, -1, -1, -1, -1]

----
i = 2, height = 8

top height = 2

2 < 8 ? yes

pop index 1

stack = [0]
stack heights = [6]

next_higher[1] = 8
next_higher = [-1, 8, -1, -1, -1]

top height = 6

6 < 8 ? yes

pop index 0

stack = []
stack heights = []

next_higher[0] = 8
next_higher = [8, 8, -1, -1, -1]

push index 2

stack = [2]
stack heights = [8]
next_higher = [8, 8, -1, -1, -1]

----
i = 3, height = 5

top height = 8

8 < 5 ? no

push index 3

stack = [2, 3]
stack heights = [8, 5]
next_higher = [8, 8, -1, -1, -1]

----
i = 4, height = 7

top height = 5

5 < 7 ? yes

pop index 3

stack = [2]
stack heights = [8]

next_higher[3] = 7
next_higher = [8, 8, -1, 7, -1]

top height = 8

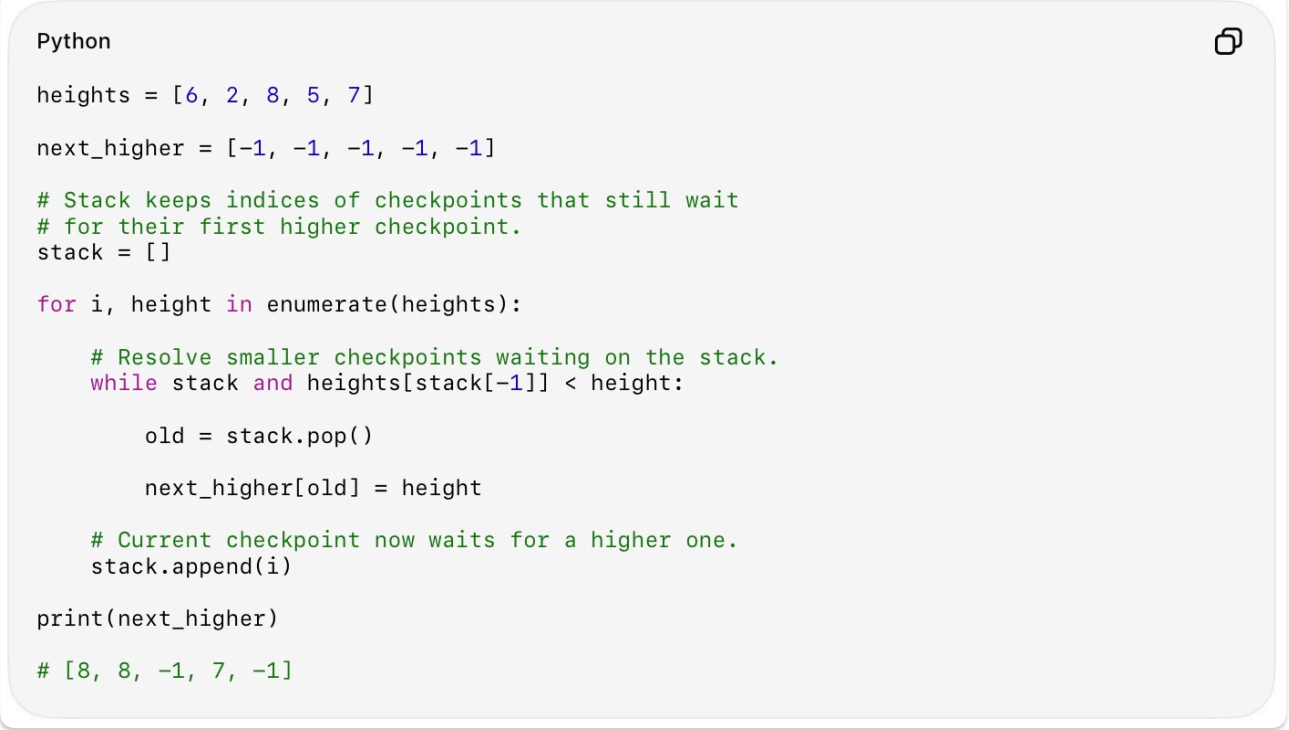
8 < 7 ? no

push index 4

stack = [2, 4]
stack heights = [8, 7]
next_higher = [8, 8, -1, 7, -1]

FINAL
stack = [2, 4]
stack heights = [8, 7]

next_higher = [8, 8, -1, 7, -1]

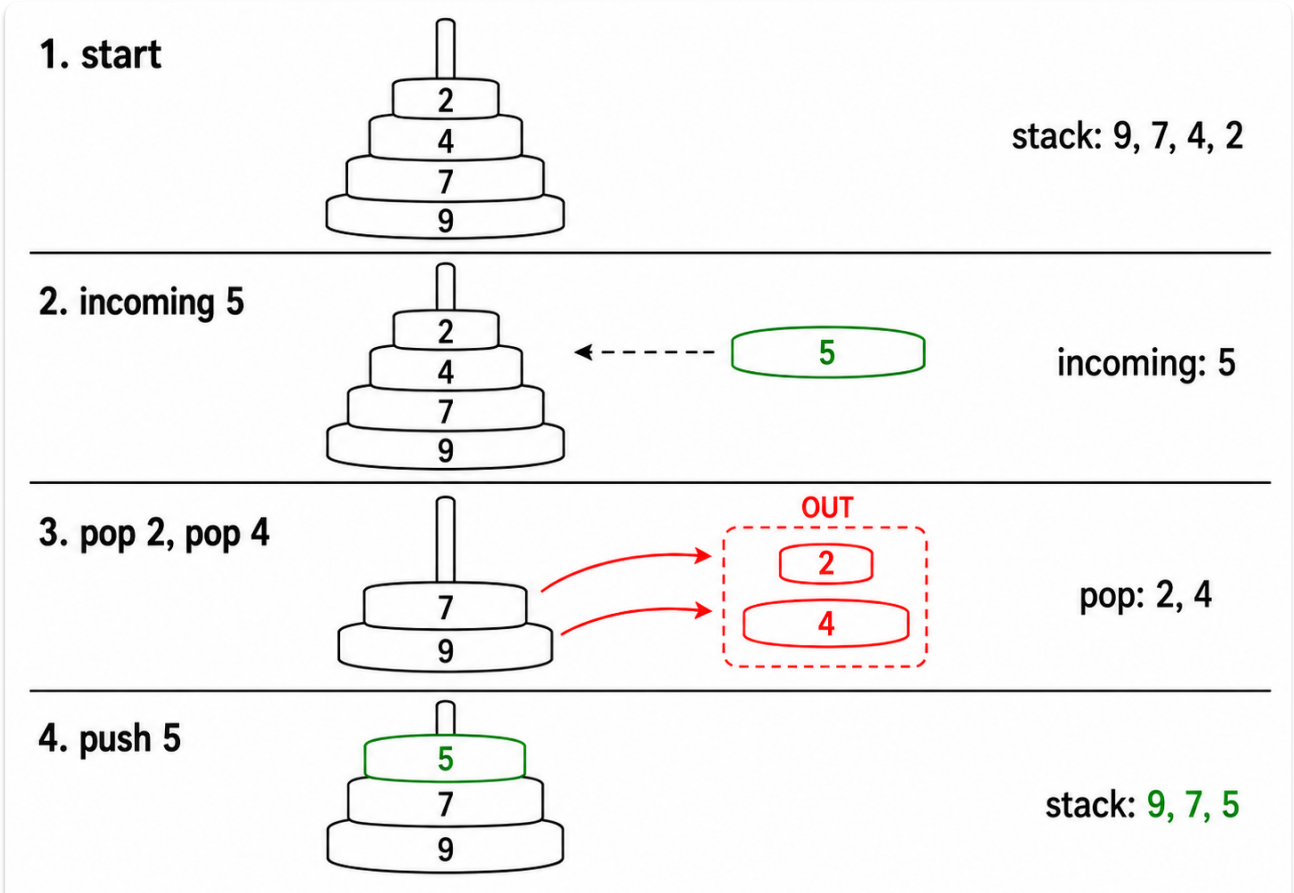


Python

```python
heights = [6, 2, 8, 5, 7]

next_higher = [-1, -1, -1, -1, -1]

# Stack keeps indices of checkpoints that still wait
# for their first higher checkpoint.
stack = []

for i, height in enumerate(heights):

    # Resolve smaller checkpoints waiting on the stack.
    while stack and heights[stack[-1]] < height:

        old = stack.pop()

        next_higher[old] = height

    # Current checkpoint now waits for a higher one.
    stack.append(i)

print(next_higher)

# [8, 8, -1, 7, -1]
```

When to think of it

- The problem asks for the **first / nearest later value** that is greater or smaller.
- I scan left to right while some previous values are still **waiting for an answer**.
- One new value can resolve **several previous values at once**.
- Once a value gets its answer, I can remove it permanently.


1. start
2
4
7
9
stack: 9, 7, 4, 2
2. incoming 5
5
incoming: 5
3. pop 2, pop 4
OUT
2
4
pop: 2, 4
4. push 5
5
7
9
stack: 9, 7, 5

A new plate removes all smaller plates from the top.